arXiv:2604.08718v1 [cs.CV] 9 Apr 2026

Accelerating Transformer-Based Monocular SLAM via Geometric Utility Scoring

Xinmiao Xiong[1*], Bangya Liu[1*], Hao Wang[2], Dayou Li[2], Nuo Chen[2], Andrew Feng[3], Mingyu Ding[4], Suman Banerjee[1], Yang Zhou[2], Zhiwen Fan[2†]
[1]UW–Madison [2]Texas A&M [3]USC [4]UNC Chapel Hill

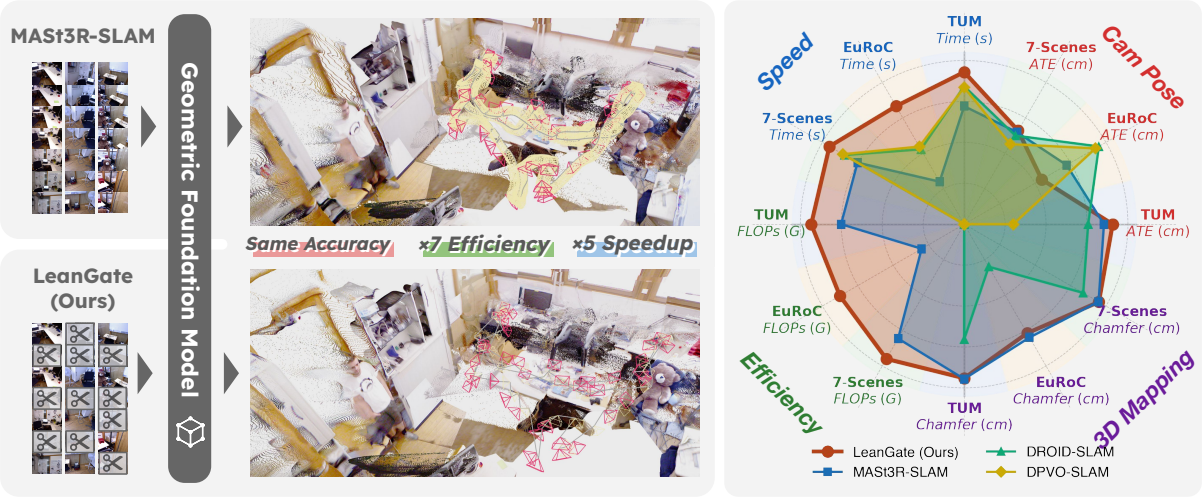

Fig. 1: We present LeanGate, a geometry-aware lightweight frame gating network that bypasses over 90% of input frames in dense streaming (red: keyframes; yellow: redundant frames), while preserving the fidelity of mapping, tracking, and camera pose estimation. We report total GPU FLOPs as the measure of efficiency, and end-to-end runtime as the measure of speedup. Since the reduction in time over the full scene is typically smaller than the overall FLOPs reduction, we report both metrics.

***Abstract*— Geometric Foundation Models (GFMs) have recently advanced monocular SLAM by providing robust, calibration-free 3D priors. However, deploying these models on dense video streams introduces significant computational redundancy. Current GFM-based SLAM systems typically rely on post-hoc keyframe selection. Because of this, they must perform expensive dense geometric decoding simply to determine if a frame contains novel geometry, resulting in late rejection and wasted computation. To mitigate this inefficiency, we propose LeanGate, a lightweight feed-forward frame-gating network. LeanGate predicts a geometric utility score to assess a frame's mapping value prior to the heavy GFM feature extraction and matching stages. By serving as a predictive plug-and-play module, our approach bypasses over 90% of redundant frames. Evaluations on standard SLAM benchmarks demonstrate that LeanGatereduces tracking FLOPs by more than 85% and achieves a 5× end-to-end throughput speedup. Furthermore, it maintains the tracking and mapping accuracy of dense baselines.**

I. Introduction

Simultaneous Localization and Mapping (SLAM) from a monocular camera serves as the spatial perception engine for modern autonomous robotics [1], [2] and augmented reality (AR) applications [3]. Traditional monocular SLAM architectures rely on multi-stage pipelines that process dense video streams, extract sparse features, and enforce hand-crafted geometric constraints to jointly estimate camera poses and 3D map structures via backend Bundle Adjustment (BA) [4], [5], [6], [7], [8], [9], [10]. Conversely, dense SLAM frameworks bypass intermediate sparse extraction to optimize photometric consistency or predict dense surfaces directly from raw pixels [11], [12], [13], [14], [15]. Both paradigms, however, demand meticulous parameter tuning and frequently degrade under challenging conditions such as weak texture, rapid sensor motion, or dynamic illumination, restricting their robustness across diverse environments.

Recently, 3D Geometric Foundation Models (GFMs) have emerged as robust, data-driven alternatives for visual perception. Models such as DUSt3R [16], MASt3R [17], and VGGT [18] take uncalibrated images and regress dense 3D representations, such as pointmaps, in a single forward pass. By learning multi-view geometric priors from massive datasets, GFMs bypass the fragility of traditional feature matching, resolving ill-posed reconstructions in textureless regions. Consequently, they offer highly stable front-ends for tracking and mapping in systems like MASt3R-SfM [19] and MASt3R-SLAM [20]. However, deploying these models on resource-constrained platforms remains challenging. Processing dense video streams (e.g., 30 FPS) at full resolution

†Corresponding author: zhiwenfan@tamu.edu. Project page: https://xinmiaoshawn.github.io/LeanGate/
*Equal contribution.

introduces severe computational redundancy, precluding real-time performance.

This computational redundancy primarily stems from an operational mismatch between how GFMs are trained and how they are deployed in SLAM. GFMs are inherently designed to recover geometry from sparse views with large baselines [21]. Yet, current GFM-based SLAM systems process dense temporal streams, incurring heavy encoding and decoding costs on nearly every frame. For instance, in MASt3R-SLAM, dense feature extraction accounts for over 50% of the runtime on a 15 FPS stream. This exposes a critical system bottleneck where keyframe selection relies on post-hoc evaluation: the system must execute the computationally expensive dense geometric decoding process simply to determine if a frame actually contains novel geometry. Consequently, this architectural flow leads to late rejection and wasted compute on highly redundant frames.

To resolve this structural inefficiency, we introduce LeanGate, a lightweight feed-forward frame-gating network. LeanGate evaluates incoming frames against a reference keyframe to predict a Geometric Utility Score, balancing geometric novelty against mapping cost. By moving the selection decision upstream prior to the heavy GFM feature extraction, LeanGate serves as a predictive gating module that filters out uninformative frames early. Empirical evaluations on standard datasets, including TUM-RGBD [22], 7-Scenes [23], and EuRoC [24], demonstrate that LeanGate bypasses over 90% of input frames while preserving the fidelity of tracking and camera pose estimation.

The primary contributions of this work are summarized as follows:

- We identify the late-rejection computational bottleneck in monocular SLAM utilizing GFM front-ends, demonstrating that the primary compute cost originates from processing temporally redundant dense streams.
- We formalize a pairwise geometric utility score and develop a lightweight feed-forward gating network, trained via distillation from a dense teacher model, to predict frame value prior to heavy geometric decoding.
- We conduct extensive evaluations across multiple SLAM benchmarks, demonstrating that LeanGate accelerates end-to-end system throughput by $5\times$ and reduces tracking FLOPs by over 85% without compromising tracking accuracy.

II. Related Work

A. Visual SLAM: From Geometry to Deep Learning

The development of Visual SLAM relies heavily on standard benchmarks such as TUM RGB-D [22], KITTI [25], and ETH3D [26], which build upon earlier evaluation efforts like SLAMBench [27]. Traditional algorithms bifurcate into indirect methods minimizing reprojection error over sparse features (PTAM [4], ORB-SLAM [5]) and direct methods optimizing photometric consistency (DSO [28], SVO [29], LSD-SLAM [30]). Because these handcrafted pipelines often fail in textureless regions or under extreme motion, deep learning initially replaced individual components with neural alternatives, utilizing SuperPoint [31] and D2-Net [32] for detection and description alongside SuperGlue [33] or LoFTR [34] for matching. Building on joint optimization approaches such as BA-Net [35] and probabilistic optimization from DeepFactors [14], DROID-SLAM integrated a differentiable Dense Bundle Adjustment layer within a recurrent framework to couple learned features with geometric solvers.

B. Foundation Models and Feed-forward Reconstruction

Geometric Foundation Models treat reconstruction as a dense regression task. Departing from incremental triangulation, DUSt3R [16] introduces a feed-forward ViT architecture inspired by RAFT [36] all-pairs correlation to regress 3D pointmaps directly from uncalibrated image pairs, implicitly inferring camera intrinsics and extrinsic poses without parametric models. This surpasses grid-based matching like GMS [37] and achieves competitive performance against kernel-based methods like DKM [38]. Unlike MVS frameworks like MVSNet [10] requiring known poses and intrinsics for cost volumes, these models infer geometry and parameters directly. MASt3R [17] extends this by learning local features alongside geometric regression. It projects dense correspondence lines from methods like PDC-Net+ [39] into 3D space with features akin to ASLFeat [40]. This 3D-centric matching outperforms classical 2D pipelines, surpasses modern matchers like LightGlue [41], and replaces learned detectors like KeyNet [42]. Although VGGT [18] offers strong priors using efficient attention akin to SegFormer [43], high memory requirements restrict it to short sequences, precluding use in long-term navigation and mapping.

C. SLAM and SfM in the Foundation Model Era

Scaling feed-forward priors for long-range trajectories remains an active challenge. MASt3R-SLAM [20] fuses two-view priors into a globally consistent system using pointmap matching and second-order optimization for calibration-free operation on unconstrained video. These GFM pipelines benchmark against COLMAP [44] and complement neural representations like iMAP [45] and NICE-SLAM [46]. Systems also explore explicit structures like EC3R [47], 3D Gaussian Splatting [48] and evaluate on challenging benchmarks like LaMAR [49] and map-free settings [50]. Unlike coordinate regression methods[51] predicting 3D coordinates directly, GFM reconstructions supply multi-view geometric priors for mapping and optimization. Aligning submaps from uncalibrated priors introduces geometric challenges. VGGT-SLAM [52] and VGGT-SLAM 2.0 [53] address projective ambiguity where uncalibrated scenes retain a 15-degree-of-freedom homography. Optimizing on the $SL(4)$ manifold corrects distortions like shear and stretch unresolved by $Sim(3)$. Transitioning to projective optimization enables metric-quality reconstruction from uncalibrated priors, connecting to multiple view geometry [54] and classical bundle adjustment [55]. Concurrently, emerging approaches adopt point-based neural mapping like Point-SLAM [56] to bridge reconstruction with scalable localization.

Method	desk	360	dsk2	rpy	xyz	flor	plnt	room	tdy	Avg
ORB3[57]	–	1.7	21.0	–	0.9	–	3.4	–	–	–
DPV[58]	11.2	1.8	2.9	3.0	1.0	5.7	2.1	33.0	8.4	7.6
DROID[15]	11.1	1.8	4.2	2.6	1.2	2.1	1.6	4.9	4.8	3.8
MASt3R-SLAM[20]										
Full(15FPS)	1.6	4.9	2.3	2.7	0.9	2.5	2.0	6.1	4.1	3.0
Keyframe(0.05FPS)	1.5	4.9	2.6	2.7	0.8	2.6	1.9	6.6	3.7	3.0
ΔATE	−0.1	0.0	0.3	0.1	−0.1	0.1	−0.1	0.5	−0.4	0.0

TABLE I: Absolute Trajectory Error (ATE in cm) on TUM-RGBD sequence fr1 comparing full-frame tracking at 15 FPS with keyframe-only tracking; lower is better.

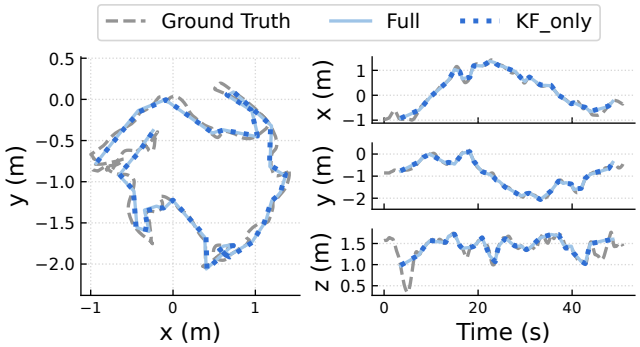

Fig. 2: $Sim(3)$-aligned trajectory comparison on TUM-RGBD *fr1-teddy*, contrasting full-frame (15 FPS) tracking with keyframe-only tracking.

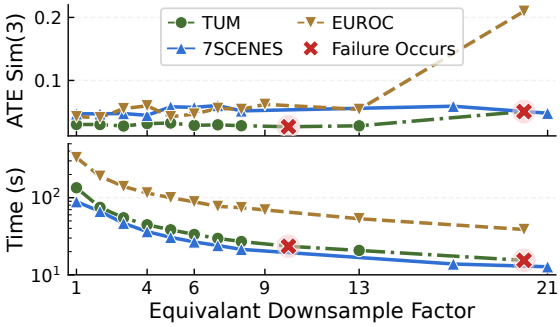

Fig. 3: ATE and runtime under naive stride policies across RGB benchmarks. The shaded region marks the stride regime where TUM starts to lose scenes; red crosses denote policies evaluated on fewer scenes than the full set.

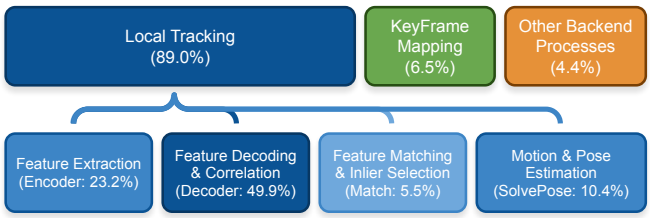

Fig. 4: Averaged per-scene time breakdown profiled on TUM-RGBD without frame/model loading time. Tracker (encoder/decoder) takes most of the computation time.

III. Analysis

A. Preliminary Experiment on Redundancy Analysis

To examine whether dense temporal processing is necessary in GFM-based SLAM, we conducted a preliminary study using MASt3R-SLAM as a representative framework. Specifically, we compare the standard full-frame tracking mode (15 FPS) with a keyframe only configuration under identical backend and optimization settings.

As shown in Table I, using dense input and using only the keyframes selected from the same dense input yield nearly identical results. This suggests that MASt3R does not actually require a large number of supportive frames to track the motion between two keyframes. For a more intuitive illustration, we visualize one representative scene in Fig. 2. The result shows that, in such a keyframe-based tracking paradigm, the 3D outputs of the keyframes alone are sufficient to recover stable and accurate camera poses.

Based on these observations, we formulate our first postulate: *Temporal redundancy in GFM-based SLAM is widespread and consistent with general RGB video characteristics.* As shown, the system maintains trajectory integrity even with aggressive frame-skipping policies.

To further validate this, we evaluated a naive stride policy across SLAM datasets. Notably, keyframes are not uniformly distributed in time, but are instead selected by the GFM based on scene coverage. As shown in Fig. 3, substantial differences already emerge across standard SLAM datasets such as TUM, 7Scenes, and EuRoC. The same stride can lead to tracking failure in some TUM sequences, degrade accuracy on EuRoC, while still being far from optimal on 7Scenes. Therefore, we formulate our second postulate: *There exists no universally optimal fixed stride for dense streaming. The ideal sampling frequency is intrinsically coupled with the scene's geometric complexity and motion dynamics.*

B. The Paradox of Post-Inference Selection

Unlike systems that update their state incrementally, such as *DROID-SLAM*, *MASt3R-SLAM* is built on a monolithic 3D reconstruction prior. In this paradigm, the metrics needed to assess a frame's geometric utility, such as spatial overlap or matching confidence, only become available **after** full dense decoding.

As illustrated in Fig. 4, this leads to a **computational paradox**: we must pay the full GPU price to decide if a frame is worth processing. The system attempts to prune redundant frames to ensure backend efficiency, yet it can only identify these redundancies by first squandering heavy resources on dense inference.

Our formulation aims to break this "process-then-evaluate" cycle. By predicting geometric gain from early-stage latent features *before* the dense decoder is invoked, we effectively decouple the selection decision from the inference cost. This distinguishes our work from the heuristic filtering in *VGGT-SLAM* and the iterative refinement in *DROID-SLAM*, providing a lean gating mechanism that ensures only informative frames reach the expensive reconstruction stage.

IV. METHODOLOGY

A. Formalization of Geometric Utility Score

We formalize the geometric utility score $\mathcal{S}$ used in the MASt3R-SLAM [20] keyframe selection pipeline and directly adopt it as the teacher signal for training LeanGate. Given an incoming frame I_i and the latest reference keyframe I_j, the model predicts dense 3D pointmaps $\mathbf{P} \in \mathbb{R}^{H\times W\times 3}$, local confidence maps $C \in [0,1]^{H\times W}$, and global quality maps $Q \in \mathbb{R}^{H\times W}$. Here, C measures the local certainty of a correspondence, while Q reflects the spatial reliability of the predicted geometry. In the following, all quantities are defined in the valid operating regime of the MASt3R-SLAM pipeline, as used throughout our experiments.

Pixel-wise validity. For each pixel $p \in \Omega_i$, the tracker establishes a correspondence $q = \mathbf{m}_{i\to j}(p)$ in I_j via an iterative re-projection search ($\mathrm{IterProj}$) that optimizes the local alignment between the predicted pointmaps $\mathbf{P}_i$ and $\mathbf{P}_j$ in the aligned comparison frame. A correspondence is considered valid if it satisfies a joint three-fold constraint:

$$\begin{aligned}\mathrm{valid}_{kf}(p) \;=\; & \big(\|\mathbf{P}_i(p) - \mathbf{P}_j(q)\|_2 < \tau_d\big) \\ & \wedge\; \big(\min(C_i(p), C_j(q)) > \tau_c\big) \\ & \wedge\; \big(\sqrt{Q_i(p)\cdot Q_j(q)} > \tau_q\big),\end{aligned} \tag{1}$$

where τ_d, τ_c, and τ_q denote the thresholds for 3D distance consistency, confidence, and spatial reliability, respectively. In particular, τ_d is measured in meters.

Frame-level utility. We aggregate pixel-wise validity into two complementary metrics. The *matching fraction* f_m measures the density of reliable constraints relative to the current frame:

$$f_m \;=\; \frac{1}{|\Omega_i|}\sum_{p\in\Omega_i} \mathbf{1}_{\{\mathrm{valid}_{kf}(p)\}}. \tag{2}$$

To quantify geometric coverage, we define the *unique fraction* f_u, which measures the proportion of the reference frame covered by these valid matches:

$$f_u = \frac{|\mathcal{U}_{i\to j}|}{|\Omega_j|}, \tag{3}$$

$$\mathcal{U}_{i\to j} = \{q \in \Omega_j \mid q = \mathbf{m}_{i\to j}(p),\; p \in \Omega_i,\; \mathrm{valid}_{kf}(p) = 1\}. \tag{4}$$

The final utility score is defined as

$$\mathcal{S} = \min(f_m, f_u), \tag{5}$$

which follows the original MASt3R-SLAM design and conservatively suppresses two failure modes: insufficient valid correspondences and insufficient geometric coverage. Following the MASt3R-SLAM indoor configuration, we set $\tau_d = 0.1\,\mathrm{m}$, $\tau_c = 0.0$, and $\tau_q = 1.5$. A new keyframe is triggered whenever $\mathcal{S} < \omega_k$ (with $\omega_k = 0.33$). We directly inherit this rule from MASt3R-SLAM, and found it to be effective and robust across all datasets used in our experiments, yielding a favorable practical trade-off between tracking stability and memory efficiency.

B. Feed-forward Score Regression

From Post-hoc Assessment to Predictive Selection. The GFM-based SLAM systems (e.g., MASt3R-SLAM) commonly adopt a post-hoc keyframe selection paradigm: the system must first run the expensive GFMs to produce dense geometric representations and perform geometric matching, and only then can it compute utility metrics like overlap. This logic leads to substantial computation and energy waste, since frames with little mapping value still trigger nearly the same peak compute path as critical keyframes.

To reduce the overhead, we move selection upstream and formulate it as a feed-forward regression problem: we predict a geometric utility score $\tau \approx 1 - \mathcal{S}$ before entering the dense reconstruction branch. Frames predicted to have low geometric utility can skip the heavy geometry branch. To instantiate this scoring mechanism, we build a lightweight utility regressor on top of FLARE's first stage and refine the utility estimate within a single forward pass, as detailed in Sec. IV-C.

C. The design of LeanGate

1) Geometric Utility Score Regressor: Instead of explicitly injecting external pose priors, we reuse the camera-aware mechanism learned inside the foundation model and build the utility regressor on top of it.

Camera-latent representation. FLARE[59] is a feed-forward model for joint camera pose estimation. We reuse the learnable camera/pose-related tokens in FLARE's decoder and their update pathway. These tokens are iteratively updated across decoder layers through the pose encoding/decoding mechanism, forming a compact latent representation of the geometric relationship for the current image pair. This decoder-layer token update is part of FLARE's internal camera-conditioning mechanism and is distinct from our iterative utility refinement head described next.

2) Iterative Refinement of Geometric Utility: To provide a more expressive prediction head for overlap estimation, we use an iterative overlap latent refinement head. For each input pair, we take the two decoder-extracted tokens, corresponding to the reference and current views, as the initial pairwise representation. Conditioned on these semantic and geometric priors, a zero-initialized overlap latent is iteratively refined by a shared trunk, and the final refined latent is used for overlap prediction.

Joint attention interaction and iterative regression. To predict the geometric utility score, we maintain a low-dimensional latent state and instantiate a latent-conditioned `score_token` at each refinement step. In the regression trunk, we concatenate the `score_token` with the pose-aware tokens and apply joint self-attention to aggregate geometric score. A lightweight prediction head then predicts an additive update to the latent utility state from the trunk output. After several refinement steps, a readout head regresses the final utility score τ from the refined latent state. Specifically, we design an *Iterative Overlap Head* that maintains a low-dimensional latent state $\mathbf{h}_{\mathrm{ov}} \in \mathbb{R}^8$. For each input pair $(I_{\mathrm{ref}}, I_{\mathrm{cur}})$, we initialize $\mathbf{h}_{\mathrm{ov}}^{(0)} = \mathbf{0}$ and perform T

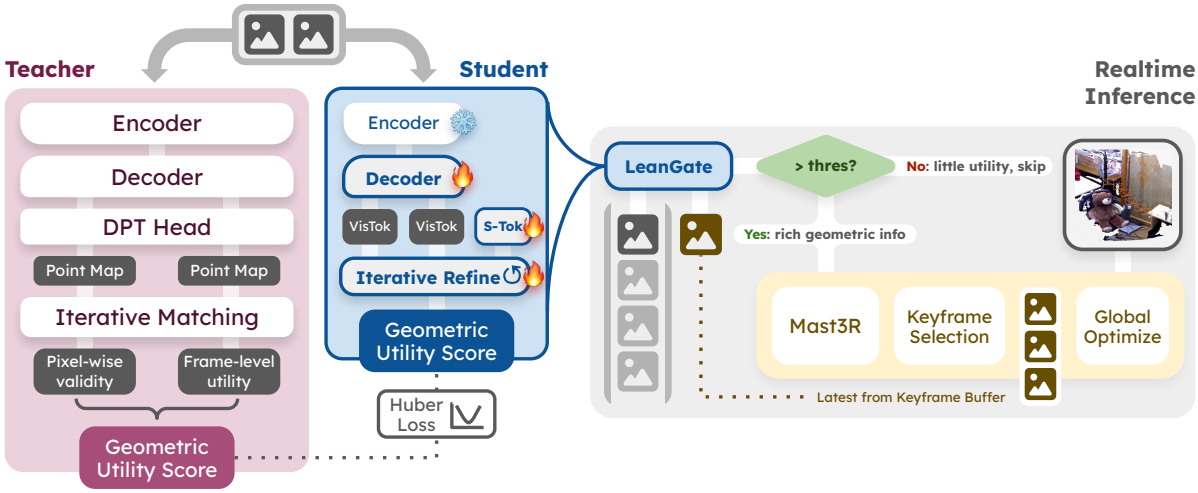

Fig. 5: System Overview for LeanGate. Left: score-only distillation. Right: MASt3R-SLAM slimmed by LeanGate. The most recent keyframe (in brown color) is fed to the LeanGate to calculate the score for the newly arriving frame (in grey).

refinement iterations on the same pairwise features, allowing the model to progressively refine its geometric score in latent space. And a system overview of LeanGate is shown in Fig. 5

3) Frame Update Policy via Utility Score: At inference time, all incoming images are handled uniformly as ordinary frames. For each frame I_{cur}, the model predicts a utility score $\tau(I_{\text{cur}})$ and applies a fixed threshold τ_{keep} to decide whether the frame should be forwarded to the SLAM system. If the score passes the threshold, the frame is kept and processed by SLAM; otherwise, it is discarded immediately. Unless otherwise specified, we use $\tau_{\text{keep}} = 0.5$ in all experiments.

D. Geometry utility score labeling with ScanNet++

Our goal in data curation is to convert static 3D environments into supervision that reflects SLAM-relevant geometric change. Instead of relying on consecutive temporal clips, we construct *geometric challenge pairs* to train the model to associate viewpoint variation with geometric utility score.

1) High-fidelity labeling.: We use ScanNet++ [60] as the primary data source for pseudo-label generation. This choice is motivated by two practical considerations. First, ScanNet++ provides high-quality reconstructions and accurate camera trajectories, which improve the reliability of correspondence-based overlap estimates. Second, ScanNet++ lies within (or close to) the training distribution of the teacher model, MASt3R [17], which helps reduce domain mismatch during pseudo-labeling. Unless otherwise specified, we run the teacher at an input resolution of 512×512 to obtain stable correspondence signals for supervision.

2) Trajectory-agnostic sampling.: Sequential sampling can allow a model to exploit temporal smoothness (e.g., near-constant velocity) rather than learning geometry. To reduce this shortcut, we sample image pairs $(\mathcal{I}_i, \mathcal{I}_j)$ based on their relative camera poses, independent of temporal adjacency. This pairwise strategy encourages the regressor to infer geometric utility from visual–spatial overlap under diverse motion patterns. We define the supervision target as

$$\tau_{\text{gt}} \;=\; 1 - \mathcal{S}(\mathcal{I}_i, \mathcal{I}_j), \tag{6}$$

where $\mathcal{S}(\cdot)$ is computed from MASt3R-SLAM's keyframe selection mechanism in Eq. 5.

E. Distillation and Inference Logic

This section describes how we transfer the teacher's geometric scoring behavior into a lightweight student suitable for real-time gating.

1) Score-only distillation.: We adopt a score-only distillation scheme: the student is trained to match the final score τ, without mimicking intermediate dense features or camera intrinsics. This design keeps training lightweight and avoids coupling the student to teacher-specific internal representations or geometric calculations.

2) Robust regression with Huber loss.: Pseudo-labels may still be noisy in visually challenging regions (e.g., strong illumination changes or weak texture). We therefore use Huber loss for regression:

$$\mathcal{L} \;=\; \frac{1}{N}\sum_{n=1}^{N} \ell_\delta\left(\tau_{\text{stu}}^{(n)} - \tau_{\text{tea}}^{(n)}\right), \tag{7}$$

with

$$\ell_\delta(a) = \begin{cases} \frac{1}{2}a^2, & |a| \le \delta, \\ \delta\left(|a| - \frac{1}{2}\delta\right), & \text{otherwise.} \end{cases} \tag{8}$$

Huber loss behaves quadratically near zero (for precise fitting) while limiting the influence of outliers.

3) Inference-time efficiency.: The student model operates independently of the teacher's dense 3D pipeline, significantly reducing computational overhead. Similar to MASt3R, it further optimizes GPU utilization by leveraging cached frames and batch inference for system-level gating.

V. Experiment

A. Experiment Setup

1) Teacher-led Dataset Generation: We construct a large-scale supervision dataset from 150 ScanNet++ scenes. To balance geometric diversity and pair density, we downsample the original 60 FPS iPhone streams to 12 FPS, yielding 0.5 M training pairs. We use an 80/20 train/eval split on scenes.

TABLE II: Reconstruction quality under different downsampling strategies on TUM RGB-D, EuRoC MAV, and 7-Scenes. We compare LeanGate against uniform striding; metrics include Completion (Comp), Chamfer distance, and F-score at 2 cm and 5 cm (higher is better for F-score, lower is better for distances). $\Delta\%$ denotes change relative to the $1\times$ baseline; LeanGate preserves quality with $16\times$–$32\times$ fewer frames, approaching the $2\times$ stride baseline.

Dataset	Method	Downsample	Comp ↓ ($\Delta\%$ ↑)	Chamfer ↓ ($\Delta\%$ ↑)	F@2cm ↑ ($\Delta\%$ ↑)	F@5cm ↑ ($\Delta\%$ ↑)
TUM RGB-D	DROID-SLAM [15]	$1\times$	0.631	0.355	0.076	0.184
	MASt3R-SLAM [20]					
	All	$1\times$	0.107	0.143	0.204	0.425
	Stride 2	$2\times$	0.129 (-20.6)	0.145 (-1.4)	0.212 (+3.9)	0.433 (+1.9)
	Stride 15	$15\times$	0.545 (-409.3)	0.393 (-174.8)	0.188 (-7.8)	0.368 (-13.4)
	LeanGate	$16\times$	0.160 (-49.5)	0.149 (-4.2)	0.202 (-1.0)	0.422 (-0.7)
EuRoC MAV	DROID-SLAM [15]	$2\times$	1.257	0.705	0.023	0.133
	MASt3R-SLAM [20]					
	All	$1\times$	0.271	0.274	0.030	0.221
	Stride 2	$2\times$	0.272 (-0.4)	0.272 (+0.7)	0.031 (+3.3)	0.224 (+1.4)
	Stride 15	$15\times$	0.370 (-36.5)	0.316 (-15.3)	0.030 (+0.0)	0.206 (-6.8)
	LeanGate	$18\times$	0.348 (-28.4)	0.298 (-8.8)	0.031 (+3.3)	0.219 (-0.9)
7-Scenes	DROID-SLAM [15]	$2\times$	0.401	0.237	0.170	0.385
	MASt3R-SLAM [20]					
	All	$1\times$	0.149	0.144	0.263	0.476
	Stride 2	$2\times$	0.150 (-0.7)	0.143 (+0.7)	0.272 (+3.4)	0.483 (+1.5)
	Stride 15	$15\times$	0.157 (-5.4)	0.143 (+0.7)	0.257 (-2.3)	0.470 (-1.3)
	LeanGate	$32\times$	0.140 (+6.0)	0.141 (+2.1)	0.264 (+0.4)	0.493 (+3.6)

TABLE III: Core results across datasets comparing trajectory accuracy and computational efficiency. For Droid-SLAM, we report both single-GPU and parallel-mode profiling on the same device using official settings.

Dataset	Model	Downsample	ATE [cm] ↓	Time [s] ↓	Calculations [TFLOPs] ↓		
					Frame Select	SLAM	Total
TUM RGB-D	DPV-SLAM [58]	$1\times$	7.6	43.79	—	—	—
	DROID-SLAM [15]	$1\times$	3.8	41.78/39.56	—	—	—
	MASt3R-SLAM [20]	$2\times$	3.00	74.95	—	6698.55	6698.55
	LeanGate	$15.58\times$	2.56	18.18	532.05	461.41	993.46
EuRoC MAV	DPV-SLAM [58]	$2\times$	2.4	122.15	—	—	—
	DROID-SLAM [15]	$2\times$	2.2	127.45/103.68	—	—	—
	MASt3R-SLAM [20]	$2\times$	4.09	189.50	—	20 835.09	20 835.09
	LeanGate	$18.60\times$	4.90	44.63	1592.16	1206.14	2798.31
7-Scenes	DPV-SLAM [58]	$2\times$	5.4	38.44	—	—	—
	DROID-SLAM [15]	$2\times$	4.9	41.29/40.75	—	—	—
	MASt3R-SLAM [20]	$2\times$	4.71	66.46	—	4978.32	4978.32
	LeanGate	$32.26\times$	4.61	12.64	318.75	174.41	493.16

For each pair $(\mathcal{I}_i, \mathcal{I}_j)$, we compute the ground-truth utility score τ_{gt} using MASt3R-SLAM, following Eq. 6. The resulting labels are approximately bell-shaped, covering a broad range from redundant pairs to visually novel viewpoints.

2) Implementation and Training: Our iterative utility regressor builds on the pretrained FLARE[61] backbone. For faster inference, we truncate the decoder from 12 to 6 layers. We re-use the selected decoder blocks and camera-conditioning modules to keep the original geometric prior and stable feature scaling. The regression head maintains an 8-D latent state and performs $K{=}4$ refinement iterations to output a scalar geometric utility score.

Training details. We supervise the model with Huber loss ($\delta{=}0.1$) for robustness to teacher labels. Training runs for 20 epochs with AdamW and decoupled learning rates: *Information Score Modules* 3×10^{-4}, *Decoder* 5×10^{-5}.

All experiments are conducted in a Docker environment (CUDA 12.4, Python 3.11) on a node with $8\times$ NVIDIA RTX A5000 (24 GB) GPUs. We use Distributed Data Parallel (DDP) with total batch size of 1024, BF16 mixed precision, and a 5-epoch linear warmup. Input images are resized to 256×256 to balance geometric detail and throughput.

B. 3D Reconstruction

We evaluate reconstruction quality using standard point-to-point metrics in Table II. Accuracy (Acc) measures the mean nearest-neighbor distance from predicted points to the reference surface (Pred→Ref), while Completeness (Comp)

measures the reverse direction (Ref→Pred). Chamfer-L1 is defined as $0.5(\text{Acc}+\text{Comp})$. We also report F-scores at 2 cm and 5 cm, where higher values indicate better reconstruction quality. All methods are evaluated in *calibrated* mode for a unified comparison protocol. Although this setting may slightly underestimate performance in some cases, it ensures consistency across methods.

Since *TUM-RGBD*, *EuRoC*, and *7-Scenes* do not provide consistent dense 3D ground truth across sequences, and depth or stereo observations are not uniformly available under a shared dense reconstruction setup, standard alternatives such as TSDF are not directly applicable in a unified evaluation. To enable consistent comparison, we use dense reconstructions generated by Map-Anything [62] as a *proxy reference geometry*. We intentionally avoid using dense MASt3R as reference to reduce bias toward methods with similar model priors. Therefore, the reported metrics reflect geometric consistency with respect to a common external proxy, rather than absolute accuracy against true 3D ground truth.

As shown in Table II, LeanGate consistently provides a stronger efficiency–quality trade-off than naive stride-based subsampling under aggressive frame reduction. On *TUM-RGBD* and *EuRoC*, it remains substantially closer to the full-frame reconstruction while clearly outperforming the high-stride baseline in both geometric distance and F-score. On *7-Scenes*, LeanGate even surpasses the full-frame setting, suggesting that our frame selection strategy can remove redundant views while preserving, and in some cases improving, reconstruction fidelity.

C. SLAM performance.

Efficiency vs. Accuracy. We evaluated LeanGate with MASt3R-SLAM [20] under identical single-threaded monocular settings. Our system achieves a $4.1\times$–$5.3\times$ end-to-end speedup by pruning over 90% redundant frames as shown in Table III. While the accuracy remains close to the baseline in most scenes, we observe small degradation in EuRoC sequences, highlighting the need for future studies on robustness at grayscale images. Additionally, we visualized the 3D trajectories for several scenes in Fig. 6. Compared with the stride-based method, our trajectories preserve finer details and adhere more closely to the true motion paths.

In standalone evaluations of SLAM tracking costs, LeanGate delivers a $5\times$–$10\times$ boost in tracking speed. This gain stems from our decoupled architecture, which enables the tracking stream to run in parallel. Furthermore, we find that LeanGate does not yet fully utilize high-end GPU resources, suggesting additional headroom for throughput scaling.

We also report FLOP counts. These are not identical to actual CUDA operation counts because MASt3R-SLAM includes backend optimizations with custom CUDA kernels that lack a standard profiling strategy. In the table, we report the best available estimates from *fvcore* and *pytorch profiling*.

D. Ablation Study

To investigate the individual contributions of our proposed components, we conduct extensive ablation studies on the ScanNet++ validation set, covering both architectural design and training strategies.

TABLE IV: Ablation on the ScanNet++ validation set for utility-score regression. The model design block evaluates architecture choices (iterative head and decoder depth), and the model training evaluates optimization choices (decoder freezing and pre-training). We report MAE and RMSE (lower is better); bold indicates the best value in each metric.

Category	Configuration	MAE ↓	RMSE ↓
Model Design			
Default	baseline (dec6)	**0.0281**	**0.0400**
Head Design	w/o iterative head	0.0352	0.0488
Decoder Depth	dec6	0.0303	0.0430
Decoder Depth	dec3	0.0347	0.0492
Model Training			
Decoder Freeze	dec12 frozen	0.0343	0.0488
Pre-training	dec3 random init	0.0537	0.0765
Pre-training	dec6 random init	0.0635	0.0883

We ablate architectural choices, focusing on the scoring head and decoder depth. Table IV shows that the scoring head dominates performance, supporting the view that iterative refinement can replace explicit geometric matching. Increasing decoder depth from *dec3* (first 3 layers) to *dec6* (first 6 layers) consistently improves final precision, suggesting that decoder-stage cross-attention is crucial for effective feature aggregation. Since iteration count is not the inference bottleneck, we set *iter*=4 (as in FLARE) to maximize accuracy.

We next study optimization choices. Table IV shows that pre-training is the dominant factor: training from random initialization leads to a substantial degradation in performance, suggesting that pre-training yields strong geometric priors that transfer to downstream refinement.

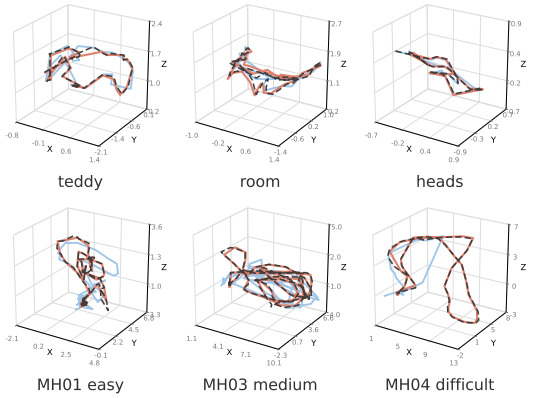

Fig. 6: Qualitative 3D trajectory comparisons on TUM-RGBD (fr1-teddy, fr1-room), 7-Scenes (heads-seq01), and EuRoC (MH01 easy, MH03 medium, MH04 difficult). Black denotes ground truth; orange shows Sim(3)-aligned estimates from LeanGate-filtered RGB streams; blue shows Sim(3)-aligned stride-15 results selected to match a similar retained-frame budget, illustrating our method's tracking consistency under aggressive frame pruning.

VI. Conclusion

In this paper, we introduced LeanGate, a lightweight feed-forward framework designed to quantify and predict frame-level information density for GFM-based systems. By identifying that high-fidelity processing is not uniformly required across all temporal observations, our work addresses a critical bottleneck in spatial tasks: the inherent computational redundancy inherent to the deployment of massive GFMs for downstream tasks such as SLAM and 3D reconstruction. Trained on $0.5\,\mathrm{M}$ samples from ScanNet++, the model already achieves strong performance across diverse indoor environments. We anticipate that further scaling of the training data will allow it to fully decouple from specific pre-training-weight dependencies and support robust acceleration for both indoor and outdoor monocular SLAM.

References

[1] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. J. Leonard, "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE T-RO*, vol. 32, no. 6, 2017.

[2] E. Marchand, H. Uchiyama, and F. Spindler, "Pose estimation for augmented reality: a hands-on survey," *IEEE TVCG*, vol. 22, no. 12, 2015.

[3] T. Schöps, J. Engel, and D. Cremers, "Semi-dense visual odometry for ar on a smartphone," in *ISMAR*, 2014, pp. 145–150.

[4] G. Klein and D. Murray, "Parallel tracking and mapping for small ar workspaces," in *ISMAR*, 2007, pp. 225–234.

[5] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "Orb-slam: A versatile and accurate monocular slam system," *IEEE T-RO*, vol. 31, no. 5, 2015.

[6] R. Mur-Artal and J. D. Tardós, "Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras," *IEEE T-RO*, vol. 33, no. 5, 2017.

[7] Y. Furukawa and C. Hernández, "Multi-view stereo: A tutorial," *FnT CGV*, vol. 9, no. 1-2, 2015.

[8] J. Engel, J. Stückler, and D. Cremers, "Large-scale direct slam with stereo cameras," in *IROS*, 2015, pp. 1935–1942.

[9] J. L. Schönberger, E. Zheng, J.-M. Frahm, and M. Pollefeys, "Pixelwise view selection for unstructured multi-view stereo," in *ECCV*, 2016, pp. 501–518.

[10] Y. Yao, Z. Luo, S. Li, T. Fang, and L. Quan, "Mvsnet: Depth inference for unstructured multi-view stereo," in *ECCV*, 2018, pp. 767–783.

[11] R. A. Newcombe, S. J. Lovegrove, and A. J. Davison, "Dtam: Dense tracking and mapping in real-time," in *ICCV*, 2011, pp. 2320–2327.

[12] H. Zhou, B. Ummenhofer, and T. Brox, "Deeptam: Deep tracking and mapping," in *ECCV*, 2018, pp. 822–838.

[13] M. Bloesch, J. Czarnowski, R. Clark, S. Leutenegger, and A. J. Davison, "Codeslam—learning a compact, optimisable representation for dense visual slam," in *CVPR*, 2018, pp. 2560–2568.

[14] J. Czarnowski, T. Laidlow, R. Clark, and A. J. Davison, "Deepfactors: Real-time probabilistic dense monocular slam," *IEEE RA-L*, vol. 5, no. 2, 2020.

[15] Z. Teed and J. Deng, "Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras," *NeurIPS*, vol. 34, pp. 16 558–16 569, 2021.

[16] S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, and J. Revaud, "Dust3r: Geometric 3d vision made easy," in *CVPR*, 2024, pp. 20 697–20 709.

[17] V. Leroy, Y. Cabon, and J. Revaud, "Grounding image matching in 3d with mast3r," in *ECCV*, 2024, pp. 71–91.

[18] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny, "Vggt: Visual geometry grounded transformer," in *CVPR*, 2025, pp. 5294–5306.

[19] B. P. Duisterhof, L. Zust, P. Weinzaepfel, V. Leroy, Y. Cabon, and J. Revaud, "Mast3r-sfm: a fully-integrated solution for unconstrained structure-from-motion," in *3DV*, 2025, pp. 1–10.

[20] R. Murai, E. Dexheimer, and A. J. Davison, "MASt3R-SLAM: Real-time dense SLAM with 3D reconstruction priors," in *CVPR*, 2025.

[21] Z. Fan, W. Cong, K. Wen, K. Wang, J. Zhang, X. Ding, D. Xu, B. Ivanovic, M. Pavone, G. Pavlakos *et al.*, "Instantsplat: Sparse-view gaussian splatting in seconds," *arXiv:2403.20309*, 2024.

[22] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of rgb-d slam systems," in *IROS*, 2012.

[23] B. Glocker, S. Izadi, J. Shotton, and A. Criminisi, "Real-time rgb-d camera relocalization," in *ISMAR*, 2013, pp. 173–179.

[24] M. Burri, J. Nikolic, P. Gohl, T. Schneider, J. Rehder, S. Omari, M. W. Achtelik, and R. Siegwart, "The euroc micro aerial vehicle datasets," *IJRR*, 2016.

[25] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The kitti dataset," *IJRR*, vol. 32, no. 11, 2013.

[26] T. Schops, T. Sattler, and M. Pollefeys, "Bad slam: Bundle adjusted direct rgb-d slam," in *CVPR*, 2019, pp. 134–144.

[27] L. Nardi, B. Bodin, M. Z. Zia, J. Mawer, A. Nisbet, P. H. Kelly, A. J. Davison, M. Luján, M. F. O'Boyle, G. Riley *et al.*, "Introducing slambench, a performance and accuracy benchmarking methodology for slam," in *ICRA*, 2015, pp. 5783–5790.

[28] J. Engel, V. Usenko, and D. Cremers, "A photometrically calibrated benchmark for monocular visual odometry," *arXiv:1607.02555*, 2016.

[29] C. Forster, M. Pizzoli, and D. Scaramuzza, "Svo: Fast semi-direct monocular visual odometry," in *ICRA*, 2014, pp. 15–22.

[30] J. Engel, T. Schöps, and D. Cremers, "Lsd-slam: Large-scale direct monocular slam," in *ECCV*, 2014, pp. 834–849.

[31] D. DeTone, T. Malisiewicz, and A. Rabinovich, "Superpoint: Self-supervised interest point detection and description," in *CVPRW*, 2018, pp. 224–236.

[32] M. Dusmanu, I. Rocco, T. Pajdla, M. Pollefeys, J. Sivic, A. Torii, and T. Sattler, "D2-net: A trainable cnn for joint description and detection of local features," in *CVPR*, 2019, pp. 8092–8101.

[33] P.-E. Sarlin, D. DeTone, T. Malisiewicz, and A. Rabinovich, "Superglue: Learning feature matching with graph neural networks," in *CVPR*, 2020, pp. 4938–4947.

[34] J. Sun, Z. Shen, Y. Wang, H. Bao, and X. Zhou, "Loftr: Detector-free local feature matching with transformers," in *CVPR*, 2021, pp. 8922–8931.

[35] C. Tang and P. Tan, "Ba-net: Dense bundle adjustment network," *arXiv:1806.04807*, 2018.

[36] Z. Teed and J. Deng, "Raft: Recurrent all-pairs field transforms for optical flow," in *ECCV*, 2020, pp. 402–419.

[37] J. Bian, W.-Y. Lin, Y. Matsushita, S.-K. Yeung, T.-D. Nguyen, and M.-M. Cheng, "Gms: Grid-based motion statistics for fast, ultra-robust feature correspondence," in *CVPR*, 2017, pp. 4181–4190.

[38] J. Edstedt, I. Athanasiadis, M. Wadenbäck, and M. Felsberg, "Dkm: Dense kernelized feature matching for geometry estimation," in *CVPR*, 2023, pp. 17 765–17 775.

[39] P. Truong, M. Danelljan, R. Timofte, and L. Van Gool, "Pdc-net+: Enhanced probabilistic dense correspondence network," *IEEE TPAMI*, vol. 45, no. 8, 2023.

[40] Z. Luo, L. Zhou, X. Bai, H. Chen, J. Zhang, Y. Yao, S. Li, T. Fang, and L. Quan, "Aslfeat: Learning local features of accurate shape and localization," in *CVPR*, 2020, pp. 6589–6598.

[41] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "Lightglue: Local feature matching at light speed," in *ICCV*, 2023, pp. 17 627–17 638.

[42] J. Barroso-Laguna, E. Riba, D. Ponsa, and K. Mikolajczyk, "Key.net: Keypoint detection by handcrafted and learned cnn filters," in *ICCV*, 2019.

[43] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "Segformer: Simple and efficient design for semantic segmentation with transformers," *NeurIPS*, vol. 34, pp. 12 077–12 090, 2021.

[44] J. L. Schonberger and J.-M. Frahm, "Structure-from-motion revisited," in *CVPR*, 2016, pp. 4104–4113.

[45] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison, "imap: Implicit mapping and positioning in real-time," in *ICCV*, 2021, pp. 6229–6238.

[46] Z. Zhu, S. Peng, V. Larsson, W. Xu, H. Bao, Z. Cui, M. R. Oswald, and M. Pollefeys, "Nice-slam: Neural implicit scalable encoding for slam," in *CVPR*, 2022, pp. 12 786–12 796.

[47] L. Hu, N. A. Oufroukh, F. Bonardi, and R. Ghandour, "Ec3r-slam: Efficient and consistent monocular dense slam with feed-forward 3d reconstruction," 2025.

[48] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis *et al.*, "3d gaussian splatting for real-time radiance field rendering." *ACM TOG*, vol. 42, no. 4, 2023.

[49] P.-E. Sarlin, M. Dusmanu, J. L. Schönberger, P. Speciale, L. Gruber, V. Larsson, O. Miksik, and M. Pollefeys, "Lamar: Benchmarking localization and mapping for augmented reality," in *ECCV*, 2022, pp. 686–704.

[50] E. Arnold, J. Wynn, S. Vicente, G. Garcia-Hernando, A. Monszpart, V. Prisacariu, D. Turmukhambetov, and E. Brachmann, "Map-free visual relocalization: Metric pose relative to a single image," in *ECCV*, 2022, pp. 690–708.

[51] E. Brachmann, T. Cavallari, and V. A. Prisacariu, "Accelerated coordinate encoding: Learning to relocalize in minutes using rgb and poses," in *CVPR*, 2023, pp. 5044–5053.

[52] D. Maggio, H. Lim, and L. Carlone, "Vggt-slam: Dense rgb slam optimized on the sl (4) manifold," *NeurIPS*, vol. 39, 2025.

[53] D. Maggio and L. Carlone, "Vggt-slam 2.0: Real time dense feed-forward scene reconstruction," *arXiv preprint arXiv:2601.19887*, 2026.

[54] R. Hartley and A. Zisserman, *Multiple view geometry in computer vision*. Cambridge university press, 2003.

[55] B. Triggs, P. F. McLauchlan, R. I. Hartley, and A. W. Fitzgibbon, "Bundle adjustment—a modern synthesis," in *Vision Algorithms*, 1999, pp. 298–372.

[56] E. Sandström, Y. Li, L. Van Gool, and M. R. Oswald, "Point-slam: Dense neural point cloud-based slam," in *ICCV*, 2023, pp. 18 433–18 444.

[57] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. Montiel, and J. D. Tardós, "Orb-slam3: An accurate open-source library for visual, visual–inertial, and multimap slam," *IEEE T-RO*, vol. 37, no. 6, 2021.

[58] L. Lipson, Z. Teed, and J. Deng, "Deep patch visual slam," in *ECCV*, 2024, pp. 424–440.

[59] S. Zhang, J. Wang, Y. Xu, N. Xue, C. Rupprecht, X. Zhou, Y. Shen, and G. Wetzstein, "Flare: Feed-forward geometry, appearance and camera estimation from uncalibrated sparse views," 2025.

[60] C. Yeshwanth, Y.-C. Liu, M. Nießner, and A. Dai, "Scannet++: A high-fidelity dataset of 3d indoor scenes," in *ICCV*, 2023, pp. 12–22.

[61] S. Zhang, J. Wang, Y. Xu, N. Xue, C. Rupprecht, X. Zhou, Y. Shen, and G. Wetzstein, "Flare: Feed-forward geometry, appearance and camera estimation from uncalibrated sparse views," in *CVPR*, 2025, pp. 21 936–21 947.

[62] N. Keetha, N. Müller, J. Schönberger, L. Porzi, Y. Zhang, T. Fischer, A. Knapitsch, D. Zauss, E. Weber, N. Antunes, J. Luiten, M. Lopez-Antequera, S. R. Bulò, C. Richardt, D. Ramanan, S. Scherer, and P. Kontschieder, "MapAnything: Universal feed-forward metric 3D reconstruction," in *International Conference on 3D Vision (3DV)*. IEEE, 2026.